

Autonomous Animation Pipeline Agents: A Blueprint for the Claude Agent SDK

This report maps six agent architectures to your existing Claude Agent SDK infrastructure — the same `query()` → skill loading → vault injection pattern you use for the Daily Driver, extended to creative production workflows. Each agent is designed around capabilities you already have: filesystem I/O, MCP tool integration, `launchd` scheduling, and the skill-as-prompt architecture from your Superuser Pack.

Agent 1: ComfyUI Sprite & Animation Orchestrator

This is the highest-leverage agent given your current blocker (sprite consistency for 16BitFit) and your career pipeline (sprites → cartoon animation → festival shorts). The ComfyUI API exposes REST endpoints for queuing prompts, polling completion, and retrieving outputs — all perfectly suited to a Python-based SDK agent.[1][2]

Architecture

The agent runs as a Python script in `agents-sdk/agents/comfyui_orchestrator.py`, triggered either by `launchd` schedule or on-demand via CLI. It uses ComfyUI's `/prompt` endpoint to queue workflow JSON, WebSocket connections to track node-by-node progress, and `/history/{prompt_id}` to retrieve completed outputs.[1]

A production-ready Python client library called **ComfyAPI** already exists on PyPI (`pip install comfyapi`) that handles workflow loading, parameter editing, batch submission, and completion polling — you wouldn't need to write the low-level HTTP/WebSocket code yourself.
[2]

Implementation Pattern

```
# agents-sdk/agents/comfyui_orchestrator.py (simplified)
from comfyapi import ComfyAPIManager
from lib.config import load_config
from lib.skill_loader import load_skills
from lib.logging_setup import setup_logger, record_run

async def run_sprite_batch(batch_config):
    manager = ComfyAPIManager()
    manager.set_base_url("http://WINDOWS_PC_IP:8188") # RTX 5080 machine
    manager.load_workflow("workflows/sprite_gen.json")

    for sprite in batch_config["sprites"]:
        manager.edit_workflow(["6", "inputs", "text"], sprite["prompt"])
        manager.edit_workflow(["3", "inputs", "seed"], sprite["seed"])
        prompt_id = manager.submit_workflow()

        while not manager.check_queue(prompt_id):
            await asyncio.sleep(2)

        # Retrieve and run vision QA
        output_url, filename = manager.find_output(prompt_id, with_filename=True)
        manager.download_output(output_url, save_path="staging/", filename=filename)

        # Vision model QA via Claude query()
        qa_result = await vision_qa(f"staging/{filename}", sprite["reference"])
        if qa_result["pass"]:
            shutil.move(f"staging/{filename}", f"approved/{sprite['name']}/")
        else:
            # Retry with adjusted parameters
            retry_queue.append(**sprite, "cfg_scale": sprite["cfg_scale"] + 0.5})
```

Key Capabilities

- **Batch queuing:** Define sprite generation batches in TOML/JSON (character, pose, animation frame), queue them sequentially or in parallel against your Windows RTX 5080 machine running ComfyUI
- **Vision-model QA:** After each generation completes, the agent calls query() with the generated image and a reference anchor image, loading your 2d-animation-principles skill as the system prompt. Claude evaluates style consistency, palette adherence, and proportional accuracy
- **Retry with parameter adjustment:** Failed QA triggers re-generation with modified parameters (higher CFG scale for tighter prompt adherence, different seed, adjusted LoRA weights)
- **TexturePacker integration:** Approved sprites are organized into folders matching TexturePacker's expected input structure. The agent then runs TexturePacker CLI to generate the final sprite sheet[3][4]

MCP Integration

Multiple ComfyUI MCP servers already exist. The alecc08/comfyui-mcp server exposes seven tools including generate_image, modify_image, resize_image, remove_background, and get_request_history — all callable through the MCP protocol. An even more feature-rich option is Peleke/comfyui-mcp which provides 37 tools covering text-to-image, ControlNet pipelines, inpainting/outpainting, IP-Adapter style transfer, and even speech synthesis. The Pixelle-MCP project from AIDC-AI goes further by converting *any* ComfyUI workflow JSON into a callable MCP tool with zero code.[5][6][7]

For your SDK agent, you'd register the ComfyUI MCP server alongside your vault tools:

```
mcp_servers={  
  "vault-tools": create_vault_mcp_server(),
```

```
"comfyui": {"type": "stdio", "command": "node", "args": ["comfyui-mcp/dist/i  
}
```

Network Consideration

Your ComfyUI runs on a Windows machine with an RTX 5080, while the Agent SDK runs on macOS. The ComfyUI API is HTTP-based, so the agent can reach it over your local network. For remote scenarios (e.g., after your Boston move), you could use Tailscale for a secure mesh — the Peleke/comfyui-mcp project explicitly documents this pattern with Tailscale + RunPod for distributed GPU compute.[6]

Agent 2: Adobe Creative Suite Bridge

This agent connects LLM decision-making to After Effects, Premiere Pro, Photoshop, and Illustrator via your existing Adobe MCP skills. The key insight: rather than just automating button clicks, the agent *reviews* compositions and *suggests* or *applies* improvements based on animation principles.

Available Infrastructure

The adb-mcp project by Mike Chambers provides a working MCP server that enables Claude to control Photoshop and Premiere Pro through UXP plugins. The architecture is: **Claude** → **MCP Server** → **Command Proxy Server** → **UXP Plugin** → **Adobe App**. This has been tested with Claude Desktop and the OpenAI Agent SDK, and the project explicitly supports Photoshop (strong) and Premiere Pro (more limited due to API constraints).[8][9]

For After Effects specifically, a Python package called after-effects-automation on PyPI provides a configuration-driven approach: you define compositions in JSON, and Python orchestrates everything through a JavaScript bridge — launching AE, building compositions, updating properties, and rendering. This is significant because it means your SDK agent can drive After Effects entirely from Python without needing a separate MCP server.[10][11]

Agent Design: Animation Review & Adjustment

```
# config.toml
[agents.ae_reviewer]
enabled = true
skills = ["2d-animation-principles", "ae-automation", "vault-read-write"]
max_turns = 40
max_budget_usd = 1.00
```

The agent loads your 2d-animation-principles skill (already translated from your NotebookLM reference library) as its system prompt. When triggered, it:

1. **Reads AE project structure** via the After Effects automation bridge or MCP server
2. **Evaluates timing** against the 12 principles of animation (squash/stretch, anticipation, slow in/slow out, etc.)
3. **Suggests or applies adjustments:** easing curves, hold frames, secondary action layers
4. **Generates motion graphics from script breakdowns:** Given a script with dialogue and stage directions, the agent creates an AE composition with text layers, timing markers, and basic motion presets

Practical Use Cases

For your animated shorts pipeline, the most impactful workflow would be **script-to-animatic**: the agent reads a script (from your vault), generates a shot breakdown with timing, creates an After Effects composition with placeholder layers at correct durations, and exports an animatic timeline. This replaces what industry benchmarks show takes 3–5 days per minute of final video when done manually.[12]

The community is actively exploring this space. One Reddit user reported using Claude to generate ExtendScript for After Effects, including a "bento box builder" and various motion design automation scripts. The consensus is that Claude handles

ExtendScript well (it's ES3-based JavaScript), but UXP scripting is less reliable due to limited training data.[13][14][15]

Recommendation: Start with ExtendScript automation (well-supported by Claude) for After Effects tasks, and use the adb-mcp UXP approach for Photoshop (where it's most mature). Your existing Adobe MCP skills that automate via ExtendScript/UXP plugins are the right foundation.

Agent 3: Festival Submission Manager

This agent tracks submission deadlines, prepares packages, and manages your festival calendar. FilmFreeway is the dominant platform for indie festival submissions, but it lacks a public API — so the agent would need a scraping + calendar hybrid approach.

Data Acquisition

A FilmFreeway scraper already exists on GitHub (pmackay/filmfreeway-scraper) that extracts festival data. For your specific needs, the agent would:[16]

1. **Maintain a festival watchlist** in your Obsidian vault at vault/20_projects/animation-pipeline/festivals.md
2. **Scrape deadline data** from FilmFreeway festival pages on a weekly schedule using Python's requests + BeautifulSoup (or Firecrawl for JS-rendered pages)
3. **Track submission status** in a structured YAML/CSV within the vault

Agent Implementation

```
# agents-sdk/agents/festival_tracker.py
# Scheduled: Every Sunday 10am via launchd

async def run(mode="weekly_scan"):
    config = load_config()
    skills = load_skills(["festival-submission", "vault-read-write"], config.skills)
```

```

# Read festival watchlist from vault
watchlist = read_vault_file("vault/20_projects/animation-pipeline/festivals.

# For each tracked festival, check deadlines
for festival in watchlist.festivals:
    deadline_data = scrape_filmfreeway_deadlines(festival.url)

    if deadline_data.next_deadline - today < timedelta(days=14):
        # Flag upcoming deadline → inject into daily note
        inject_at_anchor(
            daily_note_path(config.vault_root),
            "side-projects",
            f"△ **{festival.name}** deadline in {days_remaining} days. "
            f"Category: {festival.category}. Fee: ${festival.fee}"
        )

# Prepare submission packages for imminent deadlines
for urgent in urgent_festivals:
    package = await query(
        prompt=f"Prepare festival submission package for {urgent.name}...",
        skills=skills
    )
    # Writes synopsis, tech specs, still selections to vault

```

Submission Package Automation

Festival submissions on FilmFreeway typically require: film title, synopsis (various lengths), director's statement, tech specs (format, runtime, language, aspect ratio), production stills, poster/key art, trailer link, and a cover letter. The agent can generate or assemble all of these from your existing project files:[17]

- **Synopsis variants** (25-word, 50-word, full): Generated by Claude from your script, stored in vault
- **Tech specs**: Pulled from your Premiere Pro project metadata via the Adobe MCP bridge
- **Stills selection**: The agent scans your renders folder, uses vision to select the most visually diverse/compelling frames, and copies them to a submission-ready folder

- **Cover letter:** Generated per-festival, customized based on the festival's focus (comedy, experimental, AI film, etc.)

Calendar Integration

Using the Zapier MCP tools you already have access to (google_calendar_find_events, google_calendar_create_event), the agent can create calendar events for each deadline tier (early bird, regular, late, extended) and set reminders at 2-week and 3-day marks

.

Agent 4: Portfolio & Career Transition Engine

This agent automates the bridge between your creative output and your career transition — generating case studies from completed projects, maintaining your portfolio site, and tracking relevant job postings.

Case Study Generation

Each time you complete a project milestone (a 16BitFit feature, an animated short, a new ComfyUI workflow), the agent can generate a PM-style case study. The pattern:

1. **Trigger:** You tag a vault note with #case-study-ready or the agent detects a project status change to "completed"
2. **Research:** The agent reads all vault notes under that project's folder, extracting decisions, metrics, technical challenges, and outcomes
3. **Generation:** Using a case-study-generator skill, Claude structures the content into: Problem → Approach → Solution → Results → Learnings
4. **Output:** Markdown file in vault/30_areas/portfolio/case-studies/ + optional export to your Astro/Next.js portfolio site via Git commit

This is directly aligned with how enterprise teams are using AI for case study creation — the key is having structured source data (which your vault provides) and clear templates.[18]

Job Posting Tracker

```
# Runs weekly, scans job boards for animation/creative PM roles
async def scan_job_postings():
    search_queries = [
        "product manager animation remote",
        "product manager creative tools remote",
        "PM animation studio",
        "technical product manager video production",
    ]

    # Use web search or scraping against LinkedIn, Indeed, ZipRecruiter
    # Animation PM roles do exist remotely — Indeed shows 293+ animation
    # industry remote positions

    results = await search_multiple_boards(search_queries)

    # Filter by relevance score (Claude evaluates fit against your profile)
    ranked = await query(
        prompt=f"Rank these job postings by fit for my profile: {profile_summary}",
        context=results
    )

    # Inject top matches into weekly review
    inject_at_anchor(
        weekly_note_path(config.vault_root),
        "career-tracker",
        format_job_listings(ranked[:5])
    )
```

The animation industry is trending back toward remote/hybrid work, with many studios maintaining remote options — particularly for senior roles and those with prior studio relationships. Your unique positioning (PM skills + animation portfolio + AI pipeline expertise) targets a genuine gap in the market.[19]

Portfolio Site Maintenance

If your portfolio runs on Astro or Next.js with content sourced from markdown files (a common pattern), the agent can:

- Auto-generate project pages from vault case studies
 - Update a "Recent Work" section when new shorts are completed
 - Commit and push changes to GitHub, triggering Netlify/Vercel deployment
 - Track portfolio site analytics (via your existing Zapier → Google Sheets bridge)
-

Agent 5: Multi-Stage Production Coordinator (Director Agent)

This is the most ambitious agent — inspired directly by the **AniME framework** from Bilibili's research team, which implements a director-oriented multi-agent system for automated anime production.[20]

The AniME Architecture as a Template

AniME decomposes animation production into specialized agents coordinated by a central Director Agent that maintains a workflow graph and global Asset Memory Bank. The specialized agents include: Script & Storyboard, Character Designer, Scene Designer, Animator, Audio Production, Video Editor, and Quality Evaluator — each with dedicated MCP toolsets.[20]

The key architectural ideas you can adapt:

AniME Concept	Your SDK Implementation
Director Agent with workflow graph	A <code>production_coordinator.py</code> agent that reads a project plan from vault and dispatches sub-tasks
Asset Memory Bank (queryable database)	Your Obsidian vault with structured YAML frontmatter + folder conventions
Specialized agents with MCP toolsets	Individual SDK agents or sub-agents, each loading different skills
Structured JSON inter-agent communication	Files in a <code>vault/20_projects/{project}/pipeline/</code> directory
Quality Evaluator with vision + identity checks	Vision-model QA calls via <code>query()</code> with your animation principles skill

Practical Implementation for Your Shorts Pipeline

For a single animated short, the coordinator would manage this workflow:

1. **Script Agent** → Reads script from vault, generates shot breakdown with timing, writes to `pipeline/shot_breakdown.json`
2. **Storyboard Agent** → Takes shot breakdown, generates storyboard frames via ComfyUI (using your trained style LoRA), writes to `pipeline/storyboards/`
3. **Character Consistency Agent** → Reviews all storyboard frames for identity drift, flags inconsistencies, triggers re-generation[21]
4. **Animation Agent** → Drives After Effects composition creation via the AE automation bridge
5. **Audio Agent** → Calls ElevenLabs API for dialogue, generates timing markers
6. **Assembly Agent** → Drives Premiere Pro via MCP to assemble final edit
7. **QA Agent** → Reviews final export against original script for completeness

Each of these can be a separate SDK agent script, or you can implement them as sequential modes of a single agent (similar to how `daily_driver.py` has morning/evening/weekly modes).

Style Consistency Enforcement

Character identity drift is the #1 challenge in AI-generated animation. Your agent should implement a multi-layer consistency check:[21]

- **Anchor image comparison:** Every generation is compared against canonical character reference sheets using CLIP embeddings or direct vision-model evaluation
- **Palette enforcement:** Extract dominant colors from the anchor image and verify generated outputs stay within a defined color distance threshold
- **Proportional validation:** Check character proportions (head-to-body ratio, limb lengths) against reference measurements

The AniME framework handles this through their Asset Memory Bank, which stores "identity embeddings" alongside reference images, with single-writer semantics so only the Character Designer can update canonical identity vectors.[20]

Agent 6: Creative Production Workflow Assistants

These are smaller, focused agents that handle specific production tasks.

Storyboard-to-Animatic Agent

AI storyboard generation has matured significantly. Tools can now parse scripts into shot breakdowns with camera angle suggestions, generate consistent character frames, and even produce animatics with camera movement and timing. Your agent would:[22][12]

1. Read a script from vault
2. Use Claude to generate shot descriptions with camera direction notes (loading the 2d-animation-principles skill for shot composition guidance)

3. Generate storyboard frames via ComfyUI
4. Assemble into an animatic using After Effects automation or FFmpeg for a quick timing preview

Sound Design Coordination Agent

This agent bridges ElevenLabs (dialogue) with your editing timeline:

1. Extract dialogue lines from script with character assignments
2. Generate voice tracks via ElevenLabs API (you already have this tool)
3. Calculate timing and create audio markers
4. Import into Premiere Pro timeline via MCP

Color Grading Review Agent

A lighter agent that reviews rendered frames for color consistency:

1. Extract color palettes from key frames using Python (OpenCV/Pillow)
2. Flag frames that deviate significantly from the established palette
3. Suggest After Effects color correction adjustments
4. Write a review report to the vault for your manual review

Implementation Roadmap

Given your intermediate Python level and the existing SDK infrastructure, here's a phased approach:

Phase 1 (Weeks 1–3): ComfyUI Orchestrator

- Install comfyapi library
- Build comfyui_orchestrator.py following the Daily Driver pattern
- Implement batch sprite generation for 16BitFit
- Add vision-model QA using query() with your 2d-animation-principles skill
- Add TexturePacker CLI integration for approved sprites
- **Why first:** Directly unblocks your 16BitFit sprite consistency blocker and teaches you the core agent pattern for creative workflows

Phase 2 (Weeks 4–6): Festival Submission Tracker

- Build `festival_tracker.py` with web scraping for FilmFreeway deadlines
- Implement submission package generation
- Connect to Google Calendar via Zapier MCP
- **Why second:** You have 5 completed shorts and festival experience — this agent provides immediate career value

Phase 3 (Weeks 7–10): Adobe Bridge Agent

- Set up `adb-mcp` for Photoshop and Premiere Pro control
- Install after-effects-automation for AE pipeline
- Build `ae_reviewer.py` that evaluates compositions against animation principles
- Implement script-to-animatic workflow
- **Why third:** Depends on having ComfyUI agent working first (for storyboard frame generation)

Phase 4 (Weeks 11–14): Portfolio & Career Engine

- Build `portfolio_agent.py` for case study generation
- Implement job posting scanner
- Connect to portfolio site deployment
- **Why fourth:** Builds on outputs from all previous phases

Phase 5 (Weeks 15+): Production Coordinator

- Implement the Director Agent pattern inspired by AniME
- Coordinate multiple sub-agents for end-to-end short production
- This is the capstone — it orchestrates everything you've built

Key Technical Patterns Across All Agents

Skill Composition

Each agent loads a mix of creative and operational skills:

[\[agents.comfyui_orchestrator\]](#)

```
skills = ["sprite-generation", "2d-animation-principles", "vault-read-write"]
```

[agents.festival_tracker]

```
skills = ["festival-submission", "vault-read-write"]
```

[agents.ae_reviewer]

```
skills = ["2d-animation-principles", "ae-automation", "vault-read-write"]
```

[agents.portfolio_engine]

```
skills = ["case-study-generator", "vault-read-write"]
```

The beauty of your skill-as-prompt architecture is that improving the 2d-animation-principles skill (e.g., adding notes about timing for comedy vs. drama) automatically improves every agent that uses it .

Safety Boundaries

Creative agents need higher turn limits and budgets than the Daily Driver, but still need guardrails:

[agents.comfyui_orchestrator]

```
max_turns = 50
```

```
max_budget_usd = 2.00 # Vision QA calls add up
```

[agents.ae_reviewer]

```
max_turns = 40
```

```
max_budget_usd = 1.50
```

Keep Bash out of allowed_tools for autonomous creative agents. File I/O, vault injection, and MCP tools should be sufficient. If you need shell commands (e.g., running TexturePacker CLI), wrap them in custom MCP tools with specific command whitelists .

Vault as the Integration Layer

Every agent reads from and writes to the vault. This means your Obsidian vault becomes the central asset registry — analogous to AniME's Asset Memory Bank. Suggested vault structure additions:

[20]

```
vault/
├── 20_projects/
│   ├── 16bitfit/
│   │   ├── sprites/      ← ComfyUI agent writes approved sprites here
│   │   ├── pipeline/     ← batch configs, QA reports
│   │   └── README.md
│   ├── animation-pipeline/
│   │   ├── shorts/       ← per-short project folders
│   │   ├── festivals.md  ← festival watchlist + status tracker
│   │   ├── style-guide.md ← canonical style references
│   │   └── pipeline/     ← inter-agent communication files
│   └── portfolio/
│       ├── case-studies/ ← generated case studies
│       └── job-tracker.md ← ranked job postings
```

Cross-Machine Communication

Your setup spans macOS (agent SDK, Adobe apps) and Windows (ComfyUI + RTX 5080). The ComfyUI API is HTTP-based and works over local network. For reliability after your Boston move, consider Tailscale for a zero-config mesh VPN — this is the pattern recommended by the Peleke/comfyui-mcp project for distributed creative compute.[6]

Existing Tools and Libraries Reference

Tool	Purpose	Integration Method
comfyapi (PyPI)	ComfyUI workflow queuing + polling	Direct Python import in agent [2]
comfyui-mcp (alecc08)	7 ComfyUI tools via MCP	Register as MCP server in agent options [5]
comfyui-mcp (Peleke)	37 tools including ControlNet, IP-Adapter	MCP server + Tailscale for remote GPU [6]
Pixelles-MCP	Zero-code workflow → MCP tool conversion	MCP server [7]
adb-mcp (mikechambers)	Photoshop + Premiere control	MCP server + UXP plugins [9]
after-effects-automation (PyPI)	JSON-driven AE composition + render	Direct Python import [10]
TexturePacker CLI	Sprite sheet generation	Shell command via custom MCP tool [4]
filmfreeway-scraper	Festival data extraction	Python scraping library [16]
ElevenLabs API	Voice/dialogue generation	HTTP API calls from agent
Zapier MCP tools	Google Calendar, Slack, Sheets	Already available in your MCP config

References

1. [Hosting a ComfyUI Workflow via API - 9elements](#) - This blog post describes the basic structure of a WebSocket API that communicates with ComfyUI. Gene...

2. [SamratBarai/ComfyAPI: A python package for interacting ... - GitHub](#) - A Python client library for interacting with a running ComfyUI instance via its API. Allows you to p...
3. [How to batch-process images on Windows - CodeAndWeb](#) - First you need to add the TexturePacker's bin directory to your path. Here you find 2 versions of Te...
4. [Running TexturePacker from the command line - CodeAndWeb](#) - You can easily integrate TexturePacker into your build process and run it from command line without ...
5. [alecc08/comfyui-mcp: A simple MCP server to call text-to ... - GitHub](#) - This server provides seamless integration between MCP-compatible AI tools (like Claude Desktop) and ...
6. [Peleke/comfyui-mcp: MCP server connecting Claude to ... - GitHub](#) - Now Claude can generate images, upscale them, run ControlNet pipelines, do intelligent inpainting/ou...
7. [AIDC-AI/Pixelle-MCP: An Open-Source Multimodal AIGC ... - GitHub](#) - ☆☆☆ An AIGC solution based on the MCP protocol, supporting both local ComfyUI and cloud ComfyUI (Runni...
8. [adb-mcp MCP Server - mikechambers/adb-mcp - playbooks](#) - You can enable AI control of Adobe tools like Photoshop and Premiere by running an MCP server that e...
9. [mikechambers/adb-mcp](#) - Keep this running — the proxy server must stay active for Claude to communicate with Adobe plugins. ...
10. [after-effects-automation - PyPI](#) - Automate Adobe After Effects with Python - Create, modify, and render AE compositions programmatical...
11. [After Effects Automation is a powerful Python-based tool ... - GitHub](#) - A unified video automation platform for Adobe After Effects. Define compositions in JSON, drive them...
12. [How Can AI Animation Scene Planners Transform Your Workflow?](#) - Manual scene planning consumes 3-5 days per minute of final video, per industry benchmarks from Prod...
13. [AI that helps Artist/Non Coder make UXP plugins | Community](#) - Can adobe train an AI model to help non artist create UXP plugins as all the current AI models I tri...
14. [Turning script into plugin and struggling with Adobe UXP workflow](#) - I am trying to create a plugin that automatically stretches an adjustment layer's length based on th...

15. [Has anyone tried automating After Effects with Python + AI \(Claude ...\)](#) - I'm building a script for After Effects that uses Python + AI (Claude API) to automatically place ic...
16. [pmackay/filmfreeway-scraper: Film festivals from FilmFreeway - GitHub](#) - Film festivals from FilmFreeway. Contribute to pmackay/filmfreeway-scraper development by creating a...
17. [How much can film festivals see on your submission's filmfreeway ...](#) - Specifically wondering, on Filmfreeway, can a film festival you've submitted a film to, see what oth...
18. [AI case study generator | AI agent - Writer](#) - This AI agent helps you build compelling stories in minutes instead of hours, keeping everything cus...
19. [\[How viable is remote animation work in 2025 going into 2026? AUG ...\]](#) - My Main question to everyone: Do we feel remote animation work is still a viable option for animator...
20. [Adaptive Multi-Agent Planning for Long Animation Generation - arXiv](#) - We present AniME, a director-oriented multi-agent system for automated long-form anime production, c...
21. [AI Video's Character Consistency Problem & How to Fix It](#) - The struggle for consistent AI video is real, but it's getting solved. Understanding why models fail...
22. [How to Storyboard Your Film in Minutes Using AI \(2025 Guide\)](#) - Plan for Production: Use storyboards to inform location scouting, costume design, and shot lists — n...